

Natural Language Processing, HW1

林彦亨 111550129

1. (5%) Describe how you build your model ?

Ans:

For the RNN-based model, we implemented a Bidirectional LSTM (BiLSTM) with an attention mechanism. The architecture consists of an embedding layer (vocab size 20000, embedding dim 128), followed by a 2-layer BiLSTM (hidden dim 256, dropout 0.3), an attention pooling layer to focus on the most relevant tokens, and a fully connected output layer with sigmoid activation for multi-label classification.

For the Transformer-based model, we used *bert-base-uncased* as the pretrained backbone. The architecture takes the [CLS] token representation from the last hidden layer, applies dropout (0.3), and passes it through a linear layer to produce 12-dimensional output for multi-label classification.

(5%) How did you do to preprocess your data from dataset ? The distribution of the concern is imbalanced, what did you do to improve the macro F1 score on those concern which are in small scale ?

Ans:

1. The following preprocessing steps were applied to all tweets before training:
 - Removed URLs and @mentions, as they do not carry semantic meaning for concern classification
 - Retained hashtag text by removing only the # symbol
 - Removed special characters and extra whitespace
 - Converted all text to lowercase

For the LSTM model, a vocabulary of size 20000 was built from the training set, and each tweet was tokenized and padded to a maximum length of 80 tokens.

For the BERT model, the **BertTokenizer** was used directly with a maximum length of 128 tokens.

2. We applied two strategies to address this:

Strategy 1 — Positive class weighting (pos_weight)

We used **BCEWithLogitsLoss** with **pos_weight** to penalize the model more for missing rare labels. This gives rare labels like “**religious**” a much

higher weight, forcing the model to pay more attention to them during training.

Strategy 2 — Per-label threshold tuning

Instead of using a fixed threshold of 0.5 for all labels, we searched for the optimal threshold for each label independently on the validation set, using a grid search from 0.2 to 0.8 with step 0.02.

Labels with lower thresholds (e.g. **none**: 0.32) indicate the model's predicted probabilities for those labels tend to be low, so a lower threshold is needed to improve recall. Labels with higher thresholds (e.g. **mandatory**: 0.80) indicate the model tends to over-predict, requiring a stricter cutoff to improve precision.

2. When implementing your model:
(5%) Have you tried pretrained word embedding ? (e.g. Glove or Word2vec) What was their influence on the result after using them ?

Ans:

We experimented with GloVe (Global Vectors for Word Representation) pretrained embeddings to initialize our LSTM's embedding layer. Specifically, we used the “glove.twitter.27B.100d” version, which was trained on 27 billion tokens from Twitter — the same domain as our dataset.

We compared three settings:

Model	Val Macro-F1
LSTM (random initialization)	0.5821
LSTM + GloVe	0.6420
BERT	0.6499

Using GloVe improved the macro-F1 by **+0.060** over random initialization. We attribute this improvement to two factors. First, GloVe was trained on 27 billion tokens, providing much richer semantic knowledge than what can be learned from our 6,956 training samples alone. Second, since GloVe was trained on Twitter data specifically, it captures informal language patterns such as abbreviations and colloquial expressions commonly found in anti-vaccine tweets.

We allowed the GloVe embeddings to be fine-tuned during training, so the model could adapt the pretrained vectors to our specific task.

(5%) Have you tried attention on your model ? What was its influence on the result? When your model predict the concern, what was it focusing on ? Do some case studies.

Ans:

We added an attention pooling layer on top of the BiLSTM hidden states. Instead of using only the final hidden state to represent the entire tweet, the attention mechanism computes a weight for each token, then produces a context vector as a weighted sum.

To analyze what the model focuses on, we extracted attention weights from several validation examples:

Case 1 — side-effect:

"the vaccine gave me terrible side effects I couldn't walk for days"

The model assigned the highest attention weights to **"side"**, **"effects"**, and **"terrible"**, correctly identifying the key concern terms. Function words like "the", "me", "I" received near-zero weights.

Case 2 — ineffective:

"they cant control the Flu with vaccine so what makes them think they can control covid"

Highest attention on **"cant control"** and **"think"**, capturing the skepticism about vaccine effectiveness.

Case 3 — religious:

"God created our immune system vaccines go against his plan"

Highest attention on **"God"** and **"immune system"**, correctly linking religious language to the concern.

These case studies suggest that the attention mechanism successfully learns to focus on semantically meaningful tokens related to each concern category, rather than treating all words equally.

3. (10%) Compare the differences in performance between your RNN/LSTM model and the Transformer-based model. Which approach performs better? Explain the possible reasons for the difference.

Ans:

We compared a Bidirectional LSTM with attention pooling against a BERT-based classifier (bert-base-uncased) on the same multi-label classification task. The results are summarized below:

	BiLSTM	BERT
Val Macro-F1	0.5879	0.6499
Parameters	~5M	~110M
Training epochs	15	9
Best epoch	14	9
Convergence speed	Slow	Fast

BERT outperforms BiLSTM primarily because of its large-scale pretraining, contextual embeddings, and self-attention mechanism. The advantage is most pronounced on rare labels, where limited training data makes it impossible for BiLSTM to learn effectively. For this task, BERT is clearly the better choice when performance is the priority.